# Explaining GNN Explanations with Edge Gradients

Jesse He
jeh020@ucsd.edu
Halıcıoğlu Data Science Institute
Universiy of California, San Diego
San Diego, California, USA

Akbar Rafiey
ar9530@nyu.edu
Computer Science and Engineering
New York University
Brooklyn, New York, USA

Gal Mishne
gmishne@ucsd.edu
Halıcıoğlu Data Science Institute
Universiy of California, San Diego
San Diego, California, USA

Yusu Wang
yusuwang@ucsd.edu
Halıcıoğlu Data Science Institute
Universiy of California, San Diego
San Diego, California, USA

## Abstract

In recent years, the remarkable success of graph neural networks (GNNs) on graph-structured data has prompted a surge of methods for explaining GNN predictions. However, the state-of-the-art for GNN explainability remains in flux. Different comparisons find mixed results for different methods, with many explainers struggling on more complex GNN architectures and tasks. This presents an urgent need for a more careful theoretical analysis of competing GNN explanation methods. In this work we take a closer look at GNN explanations in two different settings: *input-level* explanations, which produce explanatory subgraphs of the input graph, and *layerwise* explanations, which produce explanatory subgraphs of the computation graph. We establish the first theoretical connections between the popular perturbation-based and classical gradient-based methods, as well as point out connections between other recently proposed methods. At the input level, we demonstrate conditions under which GNNExplainer can be approximated by a simple heuristic based on the sign of the edge gradients. In the layerwise setting, we point out that edge gradients are equivalent to occlusion search for linear GNNs. Finally, we demonstrate how our theoretical results manifest in practice with experiments on both synthetic and real datasets.

## CCS Concepts

• **Computing methodologies** → **Neural networks**; • **Computer systems organization** → **Neural networks**; • **Mathematics of computing** → *Computing most probable explanation.*

## Keywords

Graph neural networks, Explainable ML


*KDD '25, Toronto, ON, Canada.*

ACM ISBN 979-8-4007-1454-2/25/08
https://doi.org/10.1145/3711896.3736947

**ACM Reference Format:**
Jesse He, Akbar Rafiey, Gal Mishne, and Yusu Wang. 2025. Explaining GNN Explanations with Edge Gradients. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '25), August 3–7, 2025, Toronto, ON, Canada.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3711896.3736947

**KDD Availability Link:**
The source code of this paper has been made publicly available at https://doi.org/10.5281/zenodo.15485808.

## 1 Introduction

In response to the emergence of Graph Neural Networks (GNNs) for machine learning on graph-structured data, there has been a recent surge of methods aimed to *explain* GNNs. While explainability has been studied extensively in other domains [1, 5, 6, 25, 32, 35, 37], only recently have methods been proposed to deal with GNNs and the unique challenges posed by graph-structured input. In addition to the input node features, GNNs also use the given graph structure to make predictions. Therefore, explainability methods are tasked with identifying important *subgraphs*. In this work we focus on *post-hoc* explainers, which explain the predictions of a fixed, trained GNN. We also focus on *instance-based* explanation, where each prediction is explained separately, irrespective of the model's behavior on other samples. Finally, we focus on explanations in terms of the graph topology rather than the node features.

Although a number of empirical comparisons have been performed between various methods [2, 4, 10, 22], the state of the art remains unclear. The efficacy of different methods is mixed across comparisons on different tasks with different GNN models. For example, saliency maps are consistently a strong baseline, often outperforming more sophisticated methods despite their simplicity [4]. In addition, while most explainability methods are evaluated on the straightforward Graph Convolutional Network (GCN) [20], Longa et al. [22] find that other popular architectures such as Graph Isomorphism Networks (GIN) [43] are substantially more difficult for explainers. Moreover, some methods are brittle on real data despite strong performance on synthetic datasets with clear ground-truth explanations. Alarmingly, Agarwal et al. [3] find that on some datasets, existing methods may struggle to outperform even randomly selecting edges in terms of unfaithfulness.

These findings present an urgent need for a clearer theoretical understanding of GNN explainers. While some gradient and decomposition-based methods have been unified in other settings [5], the novelty of many GNN-specific methods means that there is very little unified theory. Our analysis helps both to clarify the state of explainability in GNNs and to suggest additional avenues for future research. Our contributions are as follows:

(1) We identify conditions under which GNNExplainer produces explanations which are similar to a simple gradient-based heuristic (Section 4.1).
(2) Using a decomposition of the computation graph into paths, we show that Layerwise Occlusion and Layerwise Gradients are equivalent for linear GNNs (Section 4.3).
(3) Leveraging this path decomposition, we also obtain a new interpretation of AMP-ave [41], a recent algorithm to approximate relevant paths for a GNN. We use this interpretation to describe a simple algorithm that performs an exact computation. We also draw a connection to GOAt [24], another recently proposed decomposition method (Section 5).
(4) We validate our analysis through extensive experiments on synthetic and real data. We also demonstrate that our simple algorithm for computing relevant paths outperforms AMP-ave (Section 6).

Taken together, our results show that explanability for GNNs remains closely tied to the behavior of gradient-based methods, which are simple and easy to compute. As the need for explainable GNNs continues to grow, we hope this work will provide a more unified perspective on the design space of possible explanation methods.

## 2 Related Works

Numerous explainability methods have been proposed for GNNs, and they are often often grouped into several distinct categories: *Gradient-based* methods such as saliency maps [31], Gradient⊙Input [36], or Integrated Gradients [37] use backpropagation to compute a function of the network gradients with respect to the node input features or the adjacency matrix. *Decomposition-based methods* attempt to decompose the output in terms of the inputs. They may also use backpropagation to compute attributions, as in CAM [50], LRP [6, 34], or DeepLIFT [36], or they may use other strategies based on knowledge of the network like DEGREE [12] and GOAt [24]. *Perturbation-based* methods study how the network prediction changes as the input changes. The most simple is Occlusion search [49], but recently more sophisticated methods have become popular for GNNs such as GNNExplainer [46], PGExplainer [26], and GraphMask [33]. *Search-based* methods such as SubgraphX [48], sGNN-LRP [42], and EiG-Search [23] attempt to build an explanatory subgraph by performing a heuristic search over the space of possible subgraphs. *Surrogate-based* methods like GraphLIME [17] and PGM-Explainer [39] train a simpler interpretable model which behaves similarly to the target GNN.

The plethora of competing methods has prompted several empirical evaluations, including [2, 4, 10, 22]. However, despite these empirical comparisons, their theoretical properties have only recently begun to be explored. Agarwal et al. [3] establish theoretical bounds for the faithfulness, stability, and fairness of various explainers in terms of the Lipschitz constants of the original GNN or its activation functions. Fang et al. [11] study the robustness of GNN explainers to adversarial perturbations. Similarly, Li et al. [21] find that GNN explanations are fragile when faced with adversarial attacks. Unlike our work, however, these works do not seek to establish similarities between existing GNN explanation methods.

Outside of the graph domain, more work has been done to probe different explainability methods. Ancona et al. [5] unify gradient-based methods with the decomposition methods LRP [6] and DeepLIFT [36]. Adebayo et al. [1], show that some gradient-based and decomposition methods in computer vision approximate edge detectors when applied to convolutional neural networks, implying that qualitative visual assessment of explanations can be misleading. Our contribution builds on these works by unifying explainability methods in the novel setting of GNNs.

## 3 Preliminaries

### 3.1 Graph Neural Networks

We begin by introducing some preliminary notions and notation. Throughout, let $G = (V, E)$ be an unweighted (possibly directed) graph with node features $x_v \in \mathbb{R}^p$ for each $v \in V$. We will treat undirected graphs as bidirected, with separate edges $(u, v)$ and $(v, u)$. We will denote by $\mathcal{N}(v)$ the neighborhood of a node $v$ and by $\mathcal{N}^{(k)}(v)$ the $k$-hop neighborhood of $v$ (that is, all nodes within $k$ hops of $v$). Recall that a message-passing graph neural network (GNN) consists of $L$ layers, in which each node aggregates its neighbors' embeddings and updates its own. Formally, in each layer $\ell$ the node embedding $x_v^{(\ell)}$ of node $v$ is given by

$$x_v^{(\ell)} = f_{\text{Update}}^{(\ell)}\left(f_{\text{Aggregate}}^{(\ell)}(\{x_u^{(\ell-1)} : u \in \mathcal{N}(v)\}).\right) \tag{1}$$

We will denote the output of a GNN $\Phi$ at a node $v \in G$ by $\Phi(v; G)$. A useful perspective is to consider the *computation graph* of a node $v$ with respect to an $L$-layer GNN:

*Definition 3.1.* The ($L$-layer) computation graph $G_c(v)$ of a node $v$ in a base graph $G$ is the directed graph with vertex set $V(G_c(v)) = \bigsqcup_{\ell=0}^{L} \mathcal{N}^{(\ell)}(v)$ and edges $(i^{(\ell-1)}, j^{(\ell)})$ for each $(i, j) \in E(G)$ and each $j \in \mathcal{N}^{(L-\ell)}(v)$. We refer to the set of edges $(i^{(\ell-1)}, j^{(\ell)})$ in layer $\ell$ by $E^{(\ell)}(G_c(v))$. We may write $G_c$ when $v$ is fixed.

Essentially, the computation graph $G_c(v)$ encodes the flow of information from input features into the final prediction for a specific node $v$. An important property of the computation graph $G_c$ is that it is directed and acyclic. Ordering each node $j^{(\ell)}$ of $G_c$ by its layer $\ell$ defines a topological ordering of $G_c$. We will describe how to take advantage of this property in Section 5.

### 3.2 Explanation Methods

The goal of explanation methods for GNNs is to identify an important subgraph of the input graph which explains why the network classifies a certain node with a particular label. In other words, given a GNN $\Phi$ and its prediction $Y_v$ at a node $v$ in a graph $G$, we wish to identify a subgraph $G_S$ of $G$ which explains why $\Phi$ predicts $Y_v$. We will focus on selecting subgraphs by identifying important edges, regarding node features to be of equal importance. That is, we consider attribution methods of the form $h : E(G) \to \mathbb{R}$. One can also generate explanations in the computation graph $G_c$, giving

attributions $h_c : E(G_c) \to \mathbb{R}$. By producing explanations in $G_c$, we can identify important paths that lead to the final prediction at node $v$. In this section we describe the explanation methods we will study in Section 4.

*Edge Gradients [31].* In more classical neural models such as multi-layer perceptions or convolutional networks, multiple explainability methods rely on computing the gradient of the output with respect to the input features [35–37]. In the GNN setting, one can also compute gradients with respect to the graph structure by treating the graph adjacency matrix as an input feature. Each edge $(i, j)$ with weight $\omega_{ij}$ is given the attribution

$$\textsc{Grad}_v(i, j) = \frac{\partial}{\partial \omega_{ij}} \Phi(v; G). \tag{2}$$

Since we consider unweighted graphs, we take $\omega_{ij} = 1$ for edges $(i, j)$ in the original input graph. However, our analysis can be extended to weighted graphs so long as $\omega_{ij} \geq 0$.

*Occlusion [49].* A simple perturbation-based baseline for edge attribution is *occlusion*, which studies how $\Phi(v; G)$ changes when an edge is removed. An edge $(i, j)$ is given the attribution

$$\textsc{Occ}_v(i, j) = \Phi(v; G) - \Phi(v; G \setminus (i, j)) \tag{3}$$

where $\Phi(v; G \setminus (i, j))$ is the output of the network on the perturbed graph with edge $(i, j)$ removed.

*GNNExplainer [46].* GNNExplainer is one of the earliest and most influential explainability methods designed for GNNs. Its mutual information-based objective has been adopted by [26, 27, 40]. GNNExplainer optimizes a subgraph $G_S \subset G$ which maximizes the mutual information $I(Y_v; G_S)$ between the label $Y_v$ and the subgraph $G_S$. To optimize $G_S$, GNNExplainer relaxes the problem to produce a *soft mask* $\Omega$ on $G$. Then $G_S$ is treated as a weighted graph with weight matrix $\Omega = (\omega_{ij})$, which is optimized via gradient descent. The entries $\omega_{ij}$ serve as the attributions for explanation.

In practice, optimizing the mutual information directly is difficult, so the objective is relaxed to minimizing the cross-entropy between the original prediction and the output on the masked graph $G_S$. Moreover, most implementations of GNNExplainer include regularization terms to encourage desirable properties in the final explanation. The most popular, such as in PyTorch Geometric [13], include a subgraph size constraint $\alpha \|\Omega\|_1$ to promote sparsity and an entropy constraint $\beta H(\Omega)$ to further encourage binary-valued mask entries:

$$H(\Omega) = -\frac{1}{|E|} \sum_{(i,j) \in E} (1 - \omega_{ij}) \log(1 - \omega_{ij}) + \omega_{ij} \log(\omega_{ij}) \tag{4}$$

where $\alpha, \beta$ are hyperparameters. Thus, GNNExplainer's objective is reformulated as

$$\min_{\Omega} \text{CE}(Y_v, \Phi(v; G_S(\Omega))) + \alpha \|\Omega\|_1 + \beta H(\Omega) \tag{5}$$

which is minimized using gradient descent.

# 4 Connecting Gradients and Perturbation-based Explanations

In this section, we draw theoretical connections between edge gradients and perturbation-based methods. We will consider the following explanation settings:

(1) Explanations on the *input graph*. Here, we consider the problem of selecting $G_S$ as a subgraph of the input graph $G$, as in [46]. As we will see, GNNExplainer's attributions are governed by the sign of the edge gradients.
(2) Explanations on the *computation graph*. Here, we will allow for different attributions to be given to each "copy" of an edge in the computation graph, essentially producing a subgraph for each layer of the network as in [33]. We will see that layerwise edge gradients are equivalent to layerwise occlusion for linear networks.

For simplicity, we will adopt the following setup: suppose $\Phi$ is a sum-aggregated message-passing network. We will omit bias terms, so each intermediate layer $\ell = 1, \ldots, L-1$ is given by

$$x_v^{(\ell)} = \varphi^{(\ell)} \left( \sum_{u \in \mathcal{N}(v)} W^{(\ell)} x_u^{(\ell-1)} \right) \tag{6}$$

where $\varphi^{(\ell)}$ is a (possibly nonlinear) function. The final layer has no nonlinearity:

$$\Phi(v; G) = z_v^{(L)} = \sum_{u \in \mathcal{N}(v)} W^{(L)} x_u^{(L-1)}. \tag{7}$$

Suppose also that the task is binary node classification with labels in $\{+, -\}$ where the probability of node $v$ being class $+$ is given by $\sigma(\Phi(v; G))$. That is, we apply the sigmoid $\sigma$ to the output $\Phi(v; G) \in \mathbb{R}$. For brevity, we may also write $z_v := \Phi(v; G)$. Although we consider only this simplified setting, our experiments show that this behavior is present in more complex models, multiclass classification tasks, and graph classification. Before we consider the input-level and layerwise settings separately, however, we will begin with the one-layer setting, where they coincide.

## 4.1 One-Layer Networks

Suppose $\Phi$ is a sum-aggregated linear GNN with $L = 1$ performing binary classification. As above, the probability of class $+$ is estimated with $P_\Phi(Y_v = +) = \sigma(\Phi(v; G))$. Without loss of generality, we may assume $Y_v = +$, i.e. $\Phi(v; G) > 0$. (Otherwise, we may replace $\Phi$ with $-\Phi$.) In the one-layer case, we have only one weight matrix $W$ (without a bias term). Then the underlying graph of $G_c(v)$ is simply $\mathcal{N}(v)$ and, noticing that $\frac{\partial z_v}{\partial x_u} = W$,

$$\Phi(v; G) = \sum_{u \in \mathcal{N}(v)} \frac{\partial z_v}{\partial x_u} \cdot x_u. \tag{8}$$

In this simplified one-layer case, there is a clear connection between gradient-based methods and GNNExplainer, which we prove in Appendix A.1.

Proposition 4.1. *For a GNN $\Phi$ with one linear layer, the mutual information $I(Y_v; G_S)$ is maximized when $G_S$ consists of each node $u$ and edge $(u, v)$ for which $\frac{\partial z_v}{\partial x_u} \cdot x_u > 0$.*

In particular, Proposition 4.1 tells us that in the case of a linear network with a single layer, GNNExplainer is equivalent to simply taking the sign of the edge gradients. Moreover, as shown in [5], different gradient-based attributions like Gradient⊙Input [36], 0-LRP [6], and Integrated Gradients [37] are all equivalent to each other in the one-layer setting, as well as to Occlusion.

## 4.2 Input-level Explanations

We consider the most common setting, where we wish to find an explanatory subgraph of $G$ for the prediction of the network $\Phi$ at node $v$. Here the one-layer case will be illustrative in describing the behavior of GNNExplainer: we point out that because GNNExplainer trains an edge mask via gradient descent, its optimization is governed by the edge gradients. Again, consider binary classification with labels $Y_v \in \{+, -\}$, and assume $Y_v = +$.

Proposition 4.2. *Suppose that for all $\omega_{ij} \in [0,1]$, we have $\frac{\partial z_v}{\partial \omega_{ij}} > 0$. Then*

$$\underset{\omega_{ij}^* \in [0,1]}{\operatorname{argmax}} I(Y_v \mid \omega_{ij} = \omega_{ij}^*) = 1. \tag{9}$$

*Similarly, if $\frac{\partial z_v}{\partial \omega_{ij}} < 0$ for all values of $\omega_{ij}$ in $[0,1]$,*

$$\underset{\omega_{ij}^* \in [0,1]}{\operatorname{argmax}} I(Y_v \mid \omega_{ij} = \omega_{ij}^*) = 0. \tag{10}$$

Proposition 4.2 has a straightforward interpretation: GNNExplainer follows the sign of the edge gradients. This explains, for example, the observation in [10] that GNNExplainer fails to distinguish negative evidence from irrelevant evidence. However, it also implies that GNNExplainer can be used to find negative evidence by changing the target label, as we will see in Section 6.1.1. While the condition that $\frac{\partial z_v}{\partial \omega_{ij}}$ is the same sign for all values of $\omega_{ij}$ is quite strong, it holds surprisingly often in practice, as we will see in Section 6. We discuss the effects of the regularization terms $\alpha \|\Omega\|_1$ and $\beta H(\Omega)$ in Appendix B.

## 4.3 Layerwise Explanations

Here, we consider the problem of selecting an explanatory subgraph of the computation graph as considered by [33, 34, 41]. This approach can identify important paths in $G$ which may not be apparent from input-level explanations. Attach to each edge $\left(i^{(\ell-1)}, j^{(\ell)}\right) \in E(G_c)$ a weight $\omega_{ij}^{(\ell)}$. Then we will analyze the contribution of an edge $(i, j)$ in a single layer $\ell$. For simplicity, we will assume that $\Phi$ is linear. That is, we take $\varphi^{(\ell)}$ to be the identity in (6). Then we may write

$$\Phi(v; G) = \sum_{u \in \mathcal{N}^{(L)}} \frac{\partial z_v}{\partial x_u} \cdot x_u. \tag{11}$$

We then decompose the computation graph into paths: writing $\Pi_u^v$ for the set of paths

$$\pi = \left(u^{(0)} = j_0, j_1, \ldots, j_{L-1}, j_L = v^{(L)}\right) \tag{12}$$

from $u^{(0)}$ to $v^{(L)}$ in $G_c$, we expand using the chain rule to write

$$\frac{\partial z_v}{\partial x_u} = \sum_{\pi \in \Pi_u^v} \left[ \prod_{\ell=1}^{L} \frac{\partial x_{j_\ell}^{(\ell)}}{\partial x_{j_{\ell-1}}^{(\ell-1)}} \right]. \tag{13}$$

This path decomposition is well-defined because $G_c$ is a DAG.

*Remark 4.3.* This path decomposition is employed at the neuron level in [9, 18]. We follow [34] in considering *node-level* walks in the computation graph.

Proposition 4.4. *For an $L$-layer linear GNN $\Phi$, the contribution of an edge $(i, j)$ in layer $\ell$ is given by*

$$C^{(\ell)}(i, j) := \frac{\partial z_v}{\partial \omega_{i,j}^{(\ell)}} = \frac{\partial z_v}{\partial x_j^{(\ell)}} \cdot \frac{\partial x_j^{(\ell)}}{\partial x_i^{(\ell-1)}} \cdot x_i^{(\ell-1)} \tag{14}$$

*in the sense that*

$$z_v = \sum_{(i,j) \in E^{(\ell)}(G_c(v))} C^{(\ell)}(i, j). \tag{15}$$

The proof of Proposition 4.4, which we provide in Appendix A.3, relies on decomposing the output of the network into distinct paths in the computation graph. As we will see in Section 5, this path decomposition is central to several methods including GNN-LRP [34] and GOAt [24].

Here the factor $x_i^{(\ell-1)}$ captures the computation graph of $i^{(\ell-1)}$, and the factor $\frac{\partial z_v}{\partial x_j^{(\ell)}}$ captures every path in the computation graph after $j^{(\ell)}$. In this manner, we can decompose $z_v$ into the contribution of each edge $\left(i^{(\ell-1)}, j^{(\ell)}\right)$ for a given layer $\ell$. The expression in (14), which is derived from $\frac{\partial z_v}{\partial \omega_{ij}^{(\ell)}}$, does not rely on the linearity of $\Phi$. Only the second part, (15), uses this assumption.

Notice that in the linear case, (14) does not depend on $\omega_{ij}^{(\ell)}$. Unlike the input-level case where even a linear model can have a nonlinear relationship to the edge weights $\omega_{ij}$, Proposition 4.4 shows that $\Phi$ relies linearly on each $\omega_{ij}^{(\ell)}$ when considered separately. This implies the following corollary, which connects edge gradients and Occlusion in the layerwise setting:

Corollary 4.5. *Layerwise edge gradients are equivalent to layerwise edge occlusion for a linear GNN.*

The second part of Proposition 4.4 also implies the Sensitivity-$n$ property for all $n$ from [5]. That is, removing any $n$ edges from layer $\ell$ will change $z_v$ by exactly the sum of their attributions. In particular, for $n = |E^{(\ell)}(G_c)|$ this is the *Completeness* property [37].

Analyzing nonlinear networks directly becomes challenging; even ReLU activation becomes difficult to characterize as the activity of each neuron may change in unpredictable ways when an edge is removed. Inactive neurons also lead to vanishing gradients, leaving us unable to determine what effect, if any, an edge might have if it were part of an active path. To deal with this, we borrow notation from GOAt [24], which refers to "activation patterns"

$$P_{j,k}^{(\ell)} = \begin{cases} \frac{(x_j^{(\ell)})_k}{(z_j^{(\ell)})_k} & (z_j^{(\ell)})_k \neq 0 \\ 0 & \text{otherwise} \end{cases} \tag{16}$$

where $(z_j^{(\ell)})_k$ is the $k$-th entry of the pre-activated representation of node $j$ at layer $\ell$, and $(x_j^{(\ell)})_k$ refers to the post-activated representation. If $\Phi$ is ReLU-activated, then

$$\frac{\partial x_{j_\ell}^{(\ell)}}{\partial x_{j_{\ell-1}}^{(\ell-1)}} = P_{j_\ell}^{(\ell)\,T} W^{(\ell)}. \tag{17}$$

We then use the following assumption from [9, 18, 44]:

Assumption 4.6. *Suppose each ReLU unit is activated independently with the same probability $\rho \in (0,1)$. That is, suppose each $P^{(\ell)}_{j,k}$ is a Bernoulli random variable with success probability $\rho$.*

Under Assumption 4.6,

$$\mathbb{E}\left[\frac{\partial x^{(\ell)}_{j_\ell}}{\partial x^{(\ell-1)}_{j_{\ell-1}}}\right] = \rho W^{(\ell)}. \tag{18}$$

That is, a nonlinear network will behave linearly in expectation. Although the second part of Proposition 4.4 no longer holds when $\Phi$ has nonlinearities, we can justify its utility under the Assumption 4.6 with the following corollary:

Corollary 4.7. *Let $\Phi$ be an $L$-layer sum-aggregated GNN with ReLU activations. Let $\widetilde{\Phi}$ be the network obtained by ignoring each ReLU unit. That is, the linear network with the same weights as $\Phi$. Denote by $\widetilde{C}^{(\ell)}(i,j)$ the contribution of edge $(i,j)$ in layer $\ell$ to $\widetilde{\Phi}$. Then under Assumption 4.6,*

$$\mathbb{E}[z_v] = \sum_{(i,j)\in E^{(\ell)}(G_c)} \rho^L \widetilde{C}^{(\ell)}(i,j). \tag{19}$$

## 5 Connections between Other Methods

Thus far we have seen that perturbation-based explainers can be recast in terms of gradient-based methods under certain assumptions. Here we point out additional connections between explanation methods. Note that we do not assume linearity in this section.

### 5.1 Search-based: EiG-Search

Here we point out a connection in the input-level setting between the recently proposed EiG-Search method [23] and the simple perturbation-based Occlusion method. EiG-Search uses two phases to produce explanations: edges are scored, as in other methods, and then used as a heuristic for a simple linear search to identify an important subgraph. EiG-Search proposes a so-called *linear gradient* score over subgraph components where the importance of an edge set $E_S$ is given by

$$s(v; E_S) = \frac{\Phi(v;G) - \Phi(v;G_S)}{|A(G) - A(G;E_S)|} \tag{20}$$

where

$$A(G;E_S)_{i,j} = \begin{cases} w_{\text{base}} & (i,j) \in E_S \\ A(G)_{i,j} & (i,j) \in E \setminus E_S \\ 0 & (i,j) \notin E \end{cases} \tag{21}$$

for a selected baseline weight value $w_{\text{base}}$. In their proposed method, however, $E_S$ is typically taken to be a single edge, and the baseline weight is taken to be zero in line with the zero baseline in [37]. In other words, $|A(G) - A(G;E_S)| = 1$, and so we have the following:

Proposition 5.1. *For an edge $(i,j)$ in an unweighted graph and baseline weight $w_{base} = 0$, we have $s(v;\{(i,j)\}) = \text{Occ}_v(i,j)$.*

Proposition 5.1 applies to any GNN, since both $s(v;E_S)$ and Occlusion are model-agnostic.

### 5.2 Decomposition-based: GNN-LRP

We turn our attention now to the layerwise setting, where we consider *layerwise relevance propagation* (LRP) for GNNs. The equivalence between gradient backpropagation and LRP is well-known [5, 34], but the application of LRP to GNNs is made difficult by the number of possible paths in the computation graph being exponential in $L$. Recently, [41] proposes a heuristic search approach to approximate the top node-level walks in $O(|V|^2\bar{p}^2L)$ time, where $\bar{p}$ is the maximum node feature size across all layers. However, we show that one can leverage the observation that the computation graph is a topologically sorted DAG to exactly compute the most relevant walks in comparable time:

Theorem 5.2. *The most relevant node-level walk in a GNN can be computed exactly in $O(L(|V|+|E|)\gamma(\bar{p}))$ time, where $\gamma(\bar{p}))$ is the runtime of any matrix multiplication algorithm.*

Theorem 5.2 applies to any GNN without skip connections [44]. We provide a proof and a detailed description of the algorithm as Algorithm 1 in Appendix A.4. Our Algorithm 1 traverses the computation graph the same manner as AMP-ave-Basic [41]. This also implies a tighter runtime bound of $O(L\bar{p}^2(|V|+|E|))$ for EMP-neu-Basic and AMP-ave-Basic in [41], which is significantly faster for large sparse graphs where $|V| \geq \bar{p}$.

By pointing out that AMP-ave-Basic uses the longest path algorithm for a topologically sorted DAG, we obtain a simple new interpretation of the algorithm. Furthermore, by leveraging the computation path decomposition, we can compute the *exact* walk scores rather than approximating by averaging the columns of the LRP matrix as in [41]. Essentially, rather than averaging over the neurons at each node as in [41], we defer the marginalization over neurons until the final layer, where the final layer gradient matrices provide the marginalization weights.

### 5.3 Decomposition-based: GOAt

Graph Output Attribution (GOAt) [24] is an analytical decomposition method that expands the GNN output into the sum of many scalar products. We will consider the GOAt formulation for the GNN given in (6) with ReLU activation, which is of the form

$$\mu = \left(\prod_{\ell=1}^{L} P^{(\ell)}_{\alpha_{\ell,0},\beta_{\ell,1}}\right)\left(\prod_{\ell=1}^{L} A^{(\ell)}_{\alpha_{\ell,0},\alpha_{\ell,1}}\right)(x_u)_k\left(\prod_{\ell=1}^{L} W^{(\ell)}_{\beta_{\ell,0},\beta_{\ell,1}}\right) \tag{22}$$

where $P^{(\ell)}$ refers to the matrix of activation patterns in layer $\ell$, $A^{(\ell)}$ to the adjacency matrix, $(x_u)_k$ to the $k$-th input feature of node $u$, and $W^{(\ell)}$ to the weight matrix of $\Phi$ in layer $\ell$. The indices $(\alpha_{\ell,0},\alpha_{\ell,1})$, $(\beta_{\ell,0},\beta_{\ell,1})$, range over matrix entries in layer $\ell$. Here we omit additional parameters and activations associated with, e.g., additional normalization layers or a final fully connected classifier. We denote an arbitrary component by $v$. We will denote by $\#\mu$ the number of *distinct* factors in $\mu$ (for example, if $\mu = x^2y$, then $\#\mu = 2$). Then each component $v$ is given the attribution

$$I_v(\Phi) = \sum_{\mu \ni v} \frac{\prod_{v'\in\mu} v}{\#\mu}. \tag{23}$$

Lu et al. [24] further adjust Equation (23) to account for the fact that activation patterns $P^{(\ell)}$ also depend on other components, and additionally isolate the contribution from bias terms in the network.

Again the decomposition of the computation graph into paths illustrates the connections between this method and another. Notice that each scalar product in (23) is exactly a *neuron-level walk* as described in [34]. Thus the summands can be thought of as GNN-GI scores from [34], which are given by

$$R_\mu^{\text{GNN-GI}} := \left( \prod_{\ell=1}^{L} \frac{\partial (x_{j_\ell})_{k_\ell}}{\partial (x_{\ell-1})_{k_{\ell-1}}} \right) \cdot (x_u)_{k_0} \tag{24}$$

for the neuron-level walk $((x_{j_\ell})_{k_\ell})_{\ell=1}^{L}$ corresponding to $\mu$. Then:

Proposition 5.3. *For a component $v$, we have*

$$I_v(\Phi) = \sum_{\mu \ni v} \frac{R_\mu^{GNN\text{-}GI}}{\#\mu}. \tag{25}$$

We will omit the rest of the analysis from [24], which adjusts for the relevance attributed to bias terms and the relevance of $v$ to activation patterns $P^{(\ell)}$. While GOAt typically requires expert knowledge of model parameters and careful computation of scalar product attributions, this connection between GOAt and GNN-GI (and hence with LRP) implies that GOAt can be computed using algorithms developed for LRP such as [41] and our Algorithm 1.

## 6 Experiments

To validate our theoretical insights, we compare the behavior of different GNN explainability methods across multiple experiments. We introduce a very simple baseline called *Positive Gradients*, which will give an edge an attribution of 1 if $\frac{\partial Y_v}{\partial \omega_{ij}} > \varepsilon$ and 0 otherwise. We use $\varepsilon = 0.001$ in Negative Evidence and $\varepsilon = 0$ elsewhere. (We do not see any appreciable effect in any experiment other than Negative Evidence, and we will explain this difference in behavior in the following subsection.) We also use a Random Edges baseline, which assigns a random attribution in $[0, 1]$ to each edge. Since GNNExplainer only provides input-level explanations, we also implement a Layerwise GNNExplainer which jointly trains a mask for each layer, similar to the unamortized version of GraphMask [33].

Through our experiments, we show that GNNExplainer can be approximated by the much simpler Positive Gradients on simple models, as suggested by Proposition 4.2. We also show that Layerwise Gradients approximate Layerwise Occlusion, as predicted by Corollary 4.5. We quantify the similarity between explanations using the cosine similarity of their edge masks and report the average and standard deviation over each example in the test set. We adopt cosine similarity to avoid the effects of differing sparsity for different instances in the same dataset and to avoid rescaling methods whose edge masks are not normalized between 0 and 1.

### 6.1 Synthetic Data

*6.1.1 Negative Evidence.* We use the negative evidence experiment from [10]. We generate 4 training graphs and one testing graph. Each graph has 2000 nodes. There are 10 red nodes and 10 blue nodes, with the rest of the nodes colored gray. A 1-layer linear sum-aggregated GNN with no bias terms is trained to predict if a node has more red or blue neighbors, achieving perfect accuracy.

*GNNExplainer keeps positive evidence.* As predicted by Proposition 4.1, GNNExplainer selects exactly the edges whose gradients are positive (Figure 1). Note that GNNExplainer and Edge Gradients differ due to negative evidence: GNNExplainer gives negative edges an attribution of zero as in Proposition 4.1. On the other hand, irrelevant edges have gradients very close to zero, and thus their mask values should stay near their initialization. However, GNNExplainer's regularization term $\alpha \|\Omega\|_1$ creates an additional negative gradient in its optimization, forcing irrelevant edges towards zero. By setting $\beta = 0$ and $\varepsilon = \alpha = 0.001$, we threshold away irrelevant edges as well as negative edges. Using this simple gradient heuristic, we recover the GNNExplainer masks almost exactly: across each test node in the graph, the average cosine similarity between edges with positive gradient and GNNExplainer is $0.9979 \pm 0.0029$.

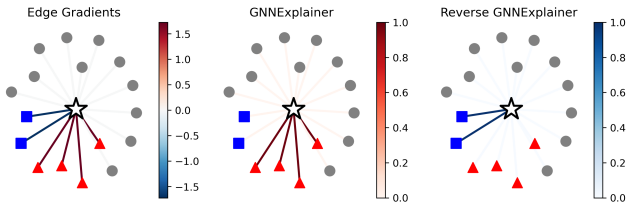


**Figure 1: Explainer predictions on Negative Evidence using Edge Gradients (left), GNNExplainer (middle), and GNNExplainer with the target class reversed (right). Edge Gradients recover the ground truth.**

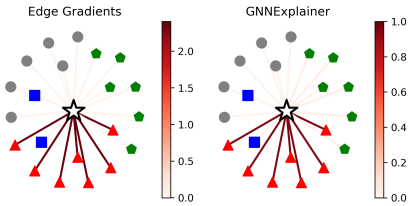


**Figure 2: Explanations on our multiclass version of Negative Evidence using Edge Gradients (left) and GNNExplainer (right). Notice that in this case, Edge Gradients does not assign negative attribution to non-red edges.**

*Negative Evidence with GNNExplainer.* Although GNNExplainer's inability to distinguish negative evidence from irrelevant evidence is potentially undesirable [10], Proposition 4.2 implies a remedy: by changing the target class for binary classification, GNNExplainer will remove the positive evidence for the original class and instead keep the negative evidence. Figure 1 shows an example where GNNExplainer matches the positive gradients and reversing the target class for GNNExplainer matches the negative gradients. In fact, subtracting the Reversed GNNExplainer explanations from the original GNNExplainer explanations gives an average of $0.9741 \pm 0.0315$ cosine similarity with the raw edge gradients.

We also perform a similar experiment using a multiclass classification task where nodes may be red, blue, or green (Figure 2). Across each test node in the graph, the average cosine similarity between edges with positive gradient and GNNExplainer is $0.9977 \pm 0.0038\%$. Notice, however, that the edge gradients no longer highlight edges to other colors as having negative contribution, as in this setting

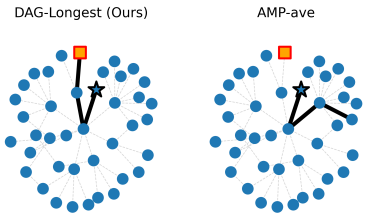


**Figure 3: Best path produced by Algorithm 1 (left) and AMP-ave-Basic (right) to explain the infection distance for the star node. Algorithm 1 recovers the ground truth, identifying 3 hops to the infected square node.**

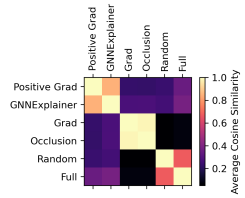


**Figure 4: Pairwise average cosine similarity between input-level explanations on Infection.**

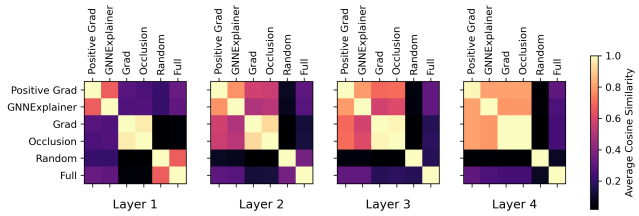


**Figure 5: Pairwise average cosine similarity between layer-wise explanations on Infection on each layer.**

the model can learn positive evidence for each class separately from negative evidence for the others.

*6.1.2 Infection.* In this task we train a 4-layer sum-aggregated GNN on the version of the infection dataset from [7, 10]. We generate random Erdős-Renyi graphs ($p = 0.004$) of 1000 nodes, each containing 50 infected nodes and 950 healthy nodes. The task is to predict the length of the shortest directed path from an infected node to the target node, with 6 classes: 0 (the node is already infected) to 5 (the node is $\geq 5$ hops from an infected node). We generate 4 training graphs and one graph for testing and explanation. The model achieves perfect accuracy on the test graph.

*Exact DAG path search outperforms AMP-ave.* Because the ground truth is in the form of paths, we evaluate our DAG-longest-path algorithm (Algorithm 1). Following [10], we explain only on nodes whose class is not 5+, as they have no ground truth explanations. We also restrict our explanations to nodes with a unique ground truth infection path to avoid ambiguity. Comparing our Algorithm 1

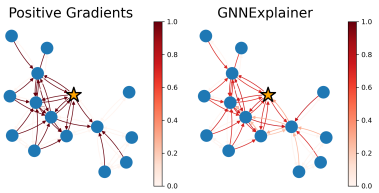


**Figure 6: Positive Gradients (left) and GNNExplainer explanation (right) for the prediction of a 2-layer GCN on a test node in Cora. The explanations have a similarity of 0.9570.**

(described in Appendix A.4) to AMP-ave-Basic [41], we demonstrate the benefit of our exact computation: Algorithm 1 successfully recovers 99.07% of the exact ground truth paths, compared to 91.12% for AMP-ave-Basic without significant difference in runtime ($8.12 \pm 5.48$s for our method vs. $8.18 \pm 5.52$s for AMP-ave). (For our comparison, we use gradients as specified in Algorithm 1, which is equivalent to 0-LRP [5, 34]. Note that we do not assume the gradients are precomputed as in [41].) We provide an example in Figure 3 where AMP-ave takes a "wrong turn" due to the approximation error in layer 3. In contrast, our exact DAG-longest path search produces the ground truth.

*Comparing Explanations.* We compare edge masks produced by six explanation methods on the test graph: Positive Gradients, GNNExplainer, Edge Gradients, Occlusion, Random Edges, and a full edge mask for the 4-hop neighborhood. Figure 4 shows the pairwise cosine similarity averaged over each example in the test set. GNNExplainer shows a high similarity to positive gradients, at $0.8472 \pm .0672$. We also examine the layerwise attributions in Figure 5. Again we see strong similarity between Positive Gradients and GNNExplainer, demonstrating the implications of Proposition 4.2. We also see that Layerwise Grad and Layerwise Occlusion have very high similarity across all layers, validating Corollary 4.5.

## 6.2 Real Data

We train three popular architectures of GNN: GCN [20], GAT [8, 38], and GIN [43]. For each architecture, we train a ReLU-activated network of $L = 1, \ldots, 5$ layers on each of five node classification datasets and three graph classification datasets. For node classification we use the citation networks Cora and Citeseer from [45] and the website networks Cornell, Texas, and Wisconsin from [30]. For graph classification we use the molecular datasets MUTAG and PROTEINS from [28], as well as Graph-SST2 from [47] We provide additional details in Appendix C.

*Comparing Explanations.* We show how Proposition 4.2 creates GNNExplainer attributions which are similar to Positive Gradients. For a qualitative example, we compare the GNNExplainer explanation for a node in Cora with the Positive Gradients explanation on the same instance (Figure 6). Quantitatively, Figure 7 shows the average cosine similarity of explanations to Positive Gradients on each GNN across all test examples. This indicates that GNNExplainer displays an insensitivity to the size of the gradients, which may account for Edge Gradients outperforming GNNExplainer on some real datasets (e.g. [4]). We also validate Corollary 4.5 across

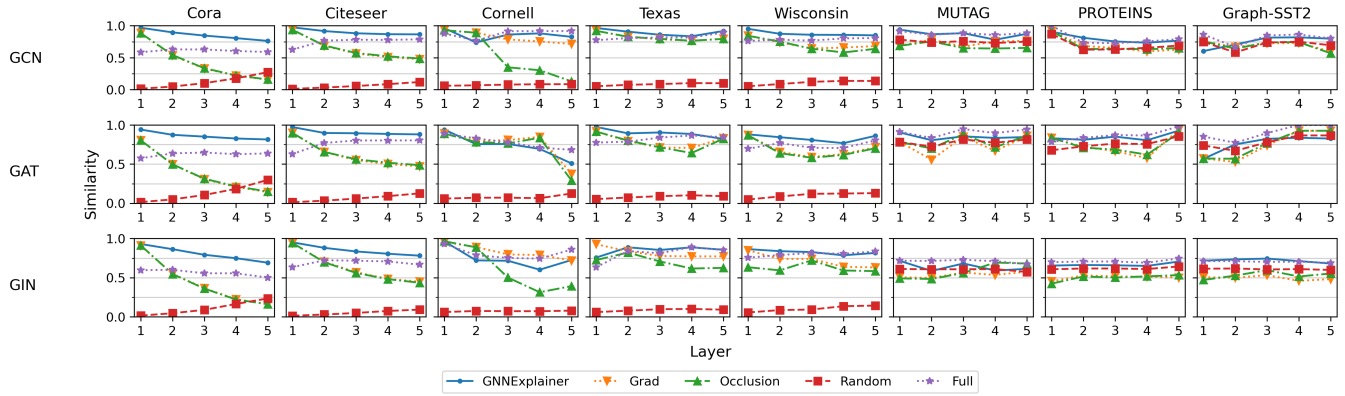


**Figure 7: Average input-level similarity between explanation methods (GNNExplainer, Edge Gradients, Occlusion, Random, and Full) and the Positive Gradients baseline. Averages are taken over each test example.**

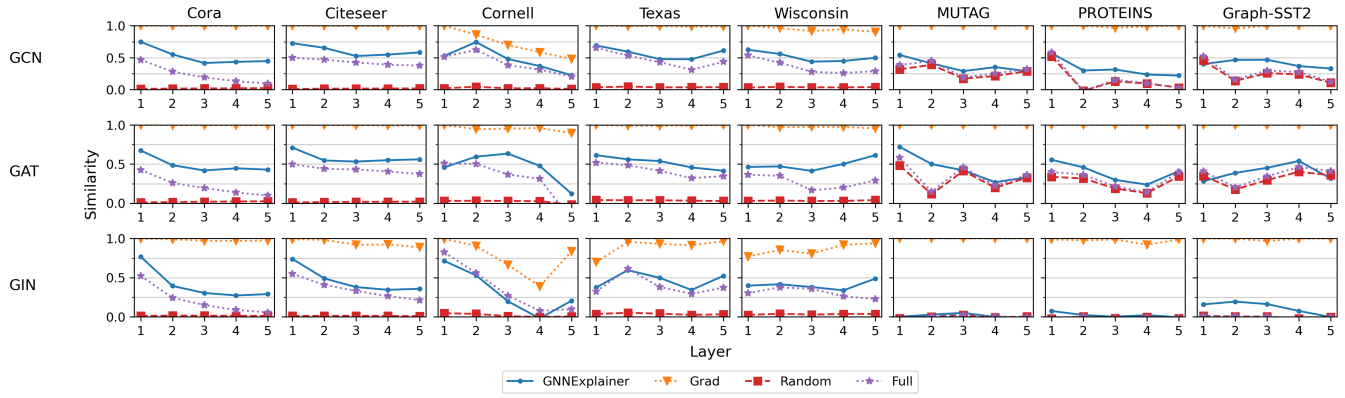


**Figure 8: Average layerwise similarity between explanation methods (GNNExplainer, Edge Gradients, Random, and Full) and Occlusion. Averages are taken over each test example and each layer.**

architectures on each dataset in Figure 8, showing the similarity of each method to Layerwise Occlusion.

*Gradients and Sign-Flipping.* We examine how often the assumption in Proposition 4.2 holds. We vary the mask values of each edge and compute the edge gradients for each test prediction and plot in Figure 9 how many edge gradients change sign across different values of $\omega_{ij} \in \{1.0, 0.95, \cdots, 0.05, 0.0\}$. For node classification, only a small percentage of edges flip, with slight increases as networks become more nonlinear. For graph classification, almost all edge gradients flip between $\omega_{ij} = 0$ and $\omega_{ij} = 1$. However, for most networks there are more flips from positive to negative than from negative to positive. Since GNNExplainer's default behavior is to promote sparse, binary edge masks with its regularization, GNNExplainer may still discard edges whose gradients flip from negative at $\omega_{ij} = 1$ to positive and keep edges with positive gradients at $\omega_{ij} = 1$, explaining the similarity we observe in Figure 7.

## 7 Conclusion

We analyze GNN explainability methods from a theoretical perspective, identifying similarities in both the input-level and layerwise settings. At the input level, our analysis reveals situations in which GNNExplainer approximates our simple Positive Gradients heuristic. In the layerwise setting, we point out that layerwise edge gradients are equivalent to layerwise occlusion in linear networks. We also point out connections between other methods, including a simple reinterpretation of AMP-ave that provides an exact algorithm with improved performance.

We note that our theoretical analysis has limitations: we mostly consider linear GNNs performing binary classification. We also consider edges individually, potentially failing to capture more intricate dependencies between edges during joint optimization. Despite these limitations, we demonstrate both of these phenomena empirically across a variety of datasets, model depths, and architectures. Future work could extend our theoretical results to more general architectures and settings, as well as analyze the behavior of other graph explanation paradigms, such as parameterized perturbation-based methods. We hope this theoretical perspective will motivate further explainability research for GNNs to leverage the ease of gradient computation, and to provide a more unified perspective on GNN explainability.

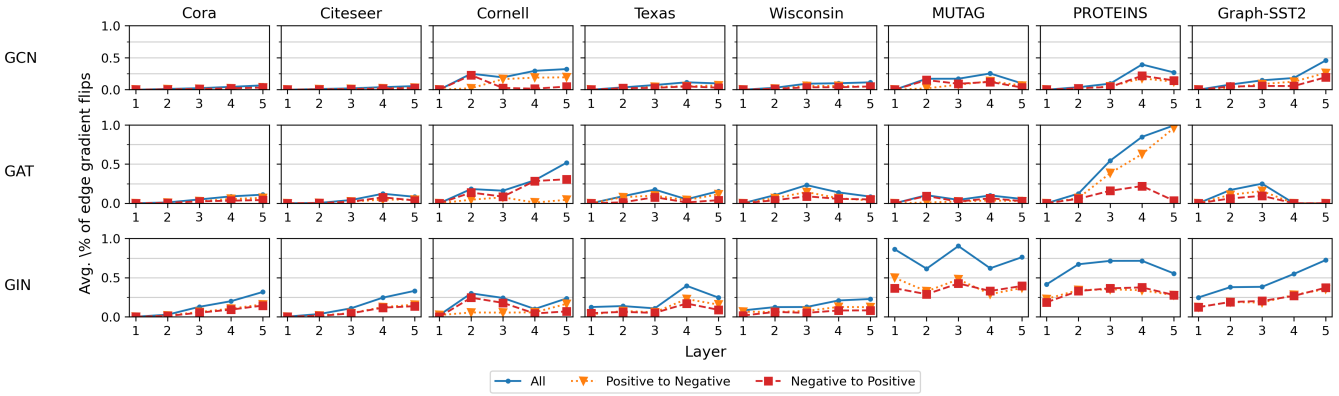


**Figure 9: Percent of edges whose gradient changes sign as edge weight varies from 1 to 0. We plot the percent of edges that show any flip, percentage of gradients that flip from positive to negative, and gradients that flip from negative to positive.**

## Acknowledgments

This work was partially supported by NSF awards CCF-2217058, 2112665, 2403452, and 2310411. J.H. was also supported by a 2024 Qualcomm Innovation Fellowship.

## References

[1] Julius Adebayo, Justin Gilmer, Michael Muelly, Ian Goodfellow, Moritz Hardt, and Been Kim. 2018. Sanity checks for saliency maps. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems* (Montréal, Canada) *(NIPS'18)*. Curran Associates Inc., Red Hook, NY, USA, 9525–9536.

[2] Chirag Agarwal, Owen Queen, Himabindu Lakkaraju, and Marinka Zitnik. 2023. Evaluating explainability for graph neural networks. *Scientific Data* 10, 1 (2023), 144.

[3] Chirag Agarwal, Marinka Zitnik, and Himabindu Lakkaraju. 2022. Probing GNN Explainers: A Rigorous Theoretical and Empirical Analysis of GNN Explanation Methods. In *Proceedings of The 25th International Conference on Artificial Intelligence and Statistics (Proceedings of Machine Learning Research, Vol. 151)*, Gustau Camps-Valls, Francisco J. R. Ruiz, and Isabel Valera (Eds.). PMLR, 8969–8996.

[4] Kenza Amara, Zhitao Ying, Zitao Zhang, Zhichao Han, Yang Zhao, Yinan Shan, Ulrik Brandes, Sebastian Schemm, and Ce Zhang. 2022. GraphFramEx: Towards Systematic Evaluation of Explainability Methods for Graph Neural Networks. In *Proceedings of the First Learning on Graphs Conference (Proceedings of Machine Learning Research, Vol. 198)*, Bastian Rieck and Razvan Pascanu (Eds.). PMLR, 44:1–44:23.

[5] Marco Ancona, Enea Ceolini, Cengiz Öztireli, and Markus Gross. 2018. Towards better understanding of gradient-based attribution methods for Deep Neural Networks. In *International Conference on Learning Representations (ICLR)*.

[6] Sebastian Bach, Alexander Binder, Grégoire Montavon, Frederick Klauschen, Klaus-Robert Müller, and Wojciech Samek. 2015. On pixel-wise explanations for non-linear classifier decisions by layer-wise relevance propagation. *PloS one* 10, 7 (2015), e0130140.

[7] Federico Baldassarre and Hossein Azizpour. 2019. Explainability Techniques for Graph Convolutional Networks. In *International Conference on Machine Learning (ICML) Workshops, 2019 Workshop on Learning and Reasoning with Graph-Structured Representations*.

[8] Shaked Brody, Uri Alon, and Eran Yahav. 2022. How Attentive are Graph Attention Networks?. In *International Conference on Learning Representations (ICLR)*.

[9] Anna Choromanska, Mikael Henaff, Michael Mathieu, Gérard Ben Arous, and Yann LeCun. 2015. The loss surfaces of multilayer networks. In *Artificial intelligence and statistics*. PMLR, 192–204.

[10] Lukas Faber, Amin K. Moghaddam, and Roger Wattenhofer. 2021. When Comparing to Ground Truth is Wrong: On Evaluating GNN Explanation Methods. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining* (Virtual Event, Singapore) *(KDD '21)*. Association for Computing Machinery, New York, NY, USA, 332–341. doi:10.1145/3447548.3467283

[11] Junfeng Fang, Wei Liu, Yuan Gao, Zemin Liu, An Zhang, Xiang Wang, and Xiangnan He. 2023. Evaluating Post-hoc Explanations for Graph Neural Networks via Robustness Analysis. In *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. Curran Associates, Inc., 72446–72463.

[12] Qizhang Feng, Ninghao Liu, Fan Yang, Ruixiang Tang, Mengnan Du, and Xia Hu. 2022. DEGREE: Decomposition Based Explanation for Graph Neural Networks. In *International Conference on Learning Representations (ICLR)*.

[13] Matthias Fey and Jan E. Lenssen. 2019. Fast Graph Representation Learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*.

[14] Will Hamilton, Zhitao Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. *Advances in neural information processing systems* 30 (2017).

[15] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2015. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the IEEE international conference on computer vision*. 1026–1034.

[16] Weihua Hu, Bowen Liu, Joseph Gomes, Marinka Zitnik, Percy Liang, Vijay Pande, and Jure Leskovec. 2020. Strategies for Pre-training Graph Neural Networks. In *International Conference on Learning Representations (ICLR)*.

[17] Qiang Huang, Makoto Yamada, Yuan Tian, Dinesh Singh, and Yi Chang. 2023. GraphLIME: Local Interpretable Model Explanations for Graph Neural Networks. *IEEE Transactions on Knowledge and Data Engineering* 35, 7 (2023), 6968–6972. doi:10.1109/TKDE.2022.3187455

[18] Kenji Kawaguchi. 2016. Deep Learning without Poor Local Minima. In *Advances in Neural Information Processing Systems*, D. Lee, M. Sugiyama, U. Luxburg, I. Guyon, and R. Garnett (Eds.), Vol. 29. Curran Associates, Inc.

[19] Diederik P Kingma. 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980* (2014).

[20] Thomas N Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations (ICLR)*.

[21] Jiate Li, Meng Pang, Yun Dong, Jinyuan Jia, and Binghui Wang. 2024. Graph neural network explanations are fragile. In *Proceedings of the 41st International Conference on Machine Learning* (Vienna, Austria) *(ICML'24)*. JMLR.org, Article 1146, 17 pages.

[22] Antonio Longa, Steve Azzolin, Gabriele Santin, Giulia Cencetti, Pietro Lio, Bruno Lepri, and Andrea Passerini. 2025. Explaining the Explainers in Graph Neural Networks: a Comparative Study. *ACM Comput. Surv.* 57, 5, Article 120 (Jan. 2025), 37 pages. doi:10.1145/3696444

[23] Shengyao Lu, Bang Liu, Keith G Mills, Jiao He, and Di Niu. 2024. EiG-Search: Generating Edge-Induced Subgraphs for GNN Explanation in Linear Time. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*, Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp (Eds.). PMLR, 33069–33088.

[24] Shengyao Lu, Keith G Mills, Jiao He, Bang Liu, and Di Niu. 2024. GOAt: Explaining Graph Neural Networks via Graph Output Attribution. In *International Conference on Learning Representations (ICLR)*.

[25] Scott M Lundberg and Su-In Lee. 2017. A unified approach to interpreting model predictions. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*. 4768–4777.

[26] Dongsheng Luo, Wei Cheng, Dongkuan Xu, Wenchao Yu, Bo Zong, Haifeng Chen, and Xiang Zhang. 2020. Parameterized Explainer for Graph Neural Network. In *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (Eds.), Vol. 33. Curran Associates, Inc., 19620–19631.

[27] Siqi Miao, Mia Liu, and Pan Li. 2022. Interpretable and generalizable graph learning via stochastic attention mechanism. In *International Conference on Machine Learning*. PMLR, 15524–15543.
[28] Christopher Morris, Nils M. Kriege, Franka Bause, Kristian Kersting, Petra Mutzel, and Marion Neumann. 2020. TUDataset: A collection of benchmark datasets for learning with graphs. arXiv:2007.08663 [cs.LG]
[29] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. *PyTorch: an imperative style, high-performance deep learning library*. Curran Associates Inc., Red Hook, NY, USA.
[30] Hongbin Pei, Bingzhe Wei, Kevin Chen-Chuan Chang, Yu Lei, and Bo Yang. 2024. Geom-GCN: Geometric Graph Convolutional Networks. In *International Conference on Learning Representations (ICLR)*.
[31] Phillip E. Pope, Soheil Kolouri, Mohammad Rostami, Charles E. Martin, and Heiko Hoffmann. 2019. Explainability Methods for Graph Convolutional Neural Networks. In *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 10764–10773. doi:10.1109/CVPR.2019.01103
[32] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. 2016. "Why should I trust you?" Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining*. 1135–1144.
[33] Michael Sejr Schlichtkrull, Nicola De Cao, and Ivan Titov. 2022. Interpreting Graph Neural Networks for NLP With Differentiable Edge Masking. In *International Conference on Learning Representations (ICLR)*.
[34] Thomas Schnake, Oliver Eberle, Jonas Lederer, Shinichi Nakajima, Kristof T Schütt, Klaus-Robert Müller, and Grégoire Montavon. 2021. Higher-order explanations of graph neural networks via relevant walks. *IEEE transactions on pattern analysis and machine intelligence* 44, 11 (2021), 7581–7596.
[35] Ramprasaath R. Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. 2017. Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization. In *2017 IEEE International Conference on Computer Vision (ICCV)*. 618–626. doi:10.1109/ICCV.2017.74
[36] Avanti Shrikumar, Peyton Greenside, and Anshul Kundaje. 2017. Learning Important Features Through Propagating Activation Differences. In *Proceedings of the 34th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 70)*, Doina Precup and Yee Whye Teh (Eds.). PMLR, 3145–3153.
[37] Mukund Sundararajan, Ankur Taly, and Qiqi Yan. 2017. Axiomatic attribution for deep networks. In *International conference on machine learning*. PMLR, 3319–3328.
[38] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. In *International Conference on Learning Representations (ICLR)*.
[39] Minh N. Vu and My T. Thai. 2020. PGM-Explainer: probabilistic graphical model explanations for graph neural networks. In *Proceedings of the 34th International Conference on Neural Information Processing Systems* (Vancouver, BC, Canada) *(NIPS '20)*. Curran Associates Inc., Red Hook, NY, USA, Article 1025, 11 pages.
[40] Tailin Wu, Hongyu Ren, Pan Li, and Jure Leskovec. 2020. Graph Information Bottleneck. In *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (Eds.), Vol. 33. Curran Associates, Inc., 20437–20448. https://proceedings.neurips.cc/paper_files/paper/2020/file/ebc2aa04e75e3caabda543a1317160c0-Paper.pdf
[41] Ping Xiong, Thomas Schnake, Michael Gastegger, Grégoire Montavon, Klaus Robert Muller, and Shinichi Nakajima. 2023. Relevant Walk Search for Explaining Graph Neural Networks. In *Proceedings of the 40th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 202)*, Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett (Eds.). PMLR, 38301–38324.
[42] Ping Xiong, Thomas Schnake, Gregoire Montavon, Klaus-Robert Müller, and Shinichi Nakajima. 2022. Efficient computation of higher-order subgraph attribution via message passing. In *International Conference on Machine Learning*. PMLR, 24478–24495.
[43] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2018. How Powerful are Graph Neural Networks?. In *International Conference on Learning Representations (ICLR)*.
[44] Keyulu Xu, Chengtao Li, Yonglong Tian, Tomohiro Sonobe, Ken-ichi Kawarabayashi, and Stefanie Jegelka. 2018. Representation Learning on Graphs with Jumping Knowledge Networks. In *Proceedings of the 35th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 80)*, Jennifer Dy and Andreas Krause (Eds.). PMLR, 5453–5462.
[45] Zhilin Yang, William Cohen, and Ruslan Salakhudinov. 2016. Revisiting semi-supervised learning with graph embeddings. In *International conference on machine learning*. PMLR, 40–48.
[46] Zhitao Ying, Dylan Bourgeois, Jiaxuan You, Marinka Zitnik, and Jure Leskovec. 2019. GNNExplainer: Generating Explanations for Graph Neural Networks. In *Advances in Neural Information Processing Systems*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett (Eds.), Vol. 32. Curran Associates, Inc.
[47] Hao Yuan, Haiyang Yu, Shurui Gui, and Shuiwang Ji. 2022. Explainability in graph neural networks: A taxonomic survey. *IEEE transactions on pattern analysis and machine intelligence* 45, 5 (2022), 5782–5799.
[48] Hao Yuan, Haiyang Yu, Jie Wang, Kang Li, and Shuiwang Ji. 2021. On Explainability of Graph Neural Networks via Subgraph Explorations. In *Proceedings of the 38th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 139)*, Marina Meila and Tong Zhang (Eds.). PMLR, 12241–12252.
[49] Matthew D Zeiler and Rob Fergus. 2014. Visualizing and understanding convolutional networks. In *Computer Vision–ECCV 2014: 13th European Conference, Zurich, Switzerland, September 6-12, 2014, Proceedings, Part I 13*. Springer, 818–833.
[50] Bolei Zhou, Aditya Khosla, Agata Lapedriza, Aude Oliva, and Antonio Torralba. 2016. Learning deep features for discriminative localization. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2921–2929.

# A Proofs

## A.1 Proof of Proposition 4.1

Proof. Following GNNExplainer, we interpret the output $Y_v|_{G_S}$ as the expectation of a Bernoulli random variable. We write the mutual information in terms of the entropy

$$I(Y_v; G_S) = H(Y_v) - H(Y_v \mid G_S) \tag{26}$$

where $H(Y_v)$ is the entropy of the (fixed) GNN prediction and $H(Y_v \mid G_S)$ is the conditional entropy of the soft mask relaxation of $G_S$. That is, each edge $(i, j)$ is assigned a Bernoulli random variable with parameter $\omega_{ij}$ and $G_S$ is sampled from the distribution over the edges defined by $\omega_{ij}$. Then we consider the problem in expectation:

$$H(Y_v \mid G_S) = \mathbb{E}\left[-\log\sigma\left(\sum_{u\in\mathcal{N}(v)} \frac{\partial z_v}{\partial x_u}\cdot x_u \cdot \mathbf{1}\{(u,v)\in G_S\}\right)\right]. \tag{27}$$

We see that $H(Y_v \mid G_S)$ is minimized when the sum $\sum_{u\in G_S} \frac{\partial z_v}{\partial x_u}\cdot x_u$ is maximized. This occurs precisely when $G_S$ contains exactly the nodes $u$ and edges $(u, v)$ for which $\frac{\partial z_v}{\partial x_u}\cdot x_u > 0$. □

*Remark A.1.* Ying et al. [46] further apply Jensen's inequality and minimize $H(Y \mid \mathbb{E}[G_S])$ where $\mathbb{E}[G_S]$ is specified by the trained soft mask $\Omega$. Despite the potential nonconvexity of $\Phi$, the authors note that the objective often produces successful explanations. In the one-layer case of Equation (27), this objective becomes

$$\min_{\Omega} H(Y \mid \mathbb{E}[G_S]) = \mathbb{E}\left[-\log\sigma\left(\sum_{u\in\mathcal{N}(v)} \frac{\partial z_v}{\partial x_u}\cdot \omega_{uv}x_u\right)\right] \tag{28}$$

from which we ecover Equation (27) by $\omega_{uv} = \mathbf{1}\{\frac{\partial z_v}{\partial x_u}\cdot x_u > 0\}$.

## A.2 Proof of Proposition 4.2

Proof. Recall that we consider binary classification and that without loss of generality, we may take $Y_v = +$. We begin by writing

$$I(Y_v \mid \omega_{ij} = \omega_{ij}^*) = H(Y_v) - H(Y_v \mid \omega_{ij} = \omega_{ij}^*) \tag{29}$$

$$= H(Y_v) - \mathbb{E}\left[-\log P_\Phi(Y_v \mid \omega_{ij}^*)\right] \tag{30}$$

$$= H(Y_v) + \mathbb{E}\left[\log\sigma(z_v|_{\omega_{ij}^*}).\right] \tag{31}$$

Thus if $\frac{\partial z_v}{\partial \omega_{ij}} > 0$ for all $\omega_{ij} \in [0, 1]$, $z_v$ is an increasing function of $\omega_{ij}$, and hence so is the mutual information. Similarly, if $\frac{\partial z_v}{\partial \omega_{ij}} < 0$

for all $\omega_{ij} \in [0,1]$, $z_v$ is a decreasing function of $\omega_{ij}$, and hence so is the mutual information. This completes the proof. □

## A.3 Proof of Proposition 4.4

Proof. We use the technique from [44] of decomposing the final output $z_v$ into each path $\pi$ in the set $\Pi_u^v$ of length-$L$ walks in $G$ which begin at a source node $u$ and end at $v$. Equivalently, $\Pi_u^v$ can be thought of as the set of maximal paths between nodes $u^{(0)}$ and $v^{(L)}$ in the computation graph $G_c$. Then we write

$$z_v = \sum_{u \in \mathcal{N}^{(L)}(v)} \left[ \sum_{\pi \in \Pi_u^v} \prod_{\ell=1}^{L} \omega_{j_{\ell-1}, j_\ell}^{(\ell)} \frac{\partial x_{j_\ell}^{(\ell)}}{\partial x_{j_{\ell-1}}^{(\ell-1)}} \right] \cdot x_u. \tag{32}$$

From the summation we can see that the predicted class logit $z_v$ can be decomposed into the contributions from each length-$L$ walk between each input node and $v$. Each path $\pi = (j_\ell)_{\ell=0}^{L}$ contributes to the prediction

$$\prod_{\ell=1}^{L} \left[ \omega_{j_{\ell-1}, j_\ell}^{(\ell)} \frac{\partial x_{j_\ell}^{(\ell)}}{\partial x_{j_{\ell-1}}^{(\ell-1)}} \right] \cdot x_{j_0}^{(0)}. \tag{33}$$

Thus we can consider the individual contribution of each edge by considering the contribution of each path which flows through that edge. Notice that in a linear network,

$$\frac{\partial z_v}{\partial \omega_{i,j}^{(\ell)}} = \sum_{\pi \ni (i^{(\ell-1)}, j^{(\ell)})} \left[ \prod_{k=\ell+1}^{L} \left( \omega_{j_{k-1}, j_k}^{(k)} \frac{\partial x_{j_k}^{(k)}}{\partial x_{j_{k-1}}^{(k-1)}} \right) \cdot \frac{\partial x_j^{(\ell)}}{\partial x_i^{(\ell-1)}} \right. \tag{34}$$

$$\left. \cdot \prod_{k=1}^{\ell-1} \left( \omega_{j_{k-1}, j_k}^{(k)} \frac{\partial x_{j_k}^{(k)}}{\partial x_{j_{k-1}}^{(k-1)}} \right) \cdot x_{j_0}^{(0)} \right] \tag{35}$$

$$= \left[ \sum_{\pi \in \Pi_j^v} \prod_{k=\ell+1}^{L} \omega_{j_{k-1}, j_k}^{(k)} \frac{\partial x_{j_k}^{(k)}}{\partial x_{j_{k-1}}^{(k-1)}} \right] \cdot \frac{\partial x_j^{(\ell)}}{\partial x_i^{(\ell-1)}} \tag{36}$$

$$\cdot \left[ \sum_{u \in \mathcal{N}^{(\ell-1)}(j)} \sum_{\pi \in \Pi_j^u} \prod_{k=1}^{\ell} \omega_{j_{k-1}, j_k}^{(k)} \frac{\partial x_{j_k}^{(k)}}{\partial x_{j_{k-1}}^{(k-1)}} \cdot x_u \right] \tag{37}$$

which, evaluated at $\omega_{j_{\ell-1}, j_\ell}^{(\ell)} = 1$ becomes

$$= \frac{\partial z_v}{\partial x_j^{(\ell)}} \cdot \frac{\partial x_j^{(\ell)}}{\partial x_i^{(\ell-1)}} \cdot x_i^{(\ell-1)} \tag{38}$$

Moreover, restricting our attention to layer $(\ell - 1)$, we see that

$$z_v = \sum_{u^{(\ell-1)} \in G_c^{(\ell-1)}} \frac{\partial z_v}{\partial x_j^{(\ell)}} \cdot \frac{\partial x_j^{(\ell)}}{\partial x_i^{(\ell-1)}} \cdot x_i^{(\ell-1)}. \tag{39}$$

□

## A.4 Proof of Theorem 5.2

Proof. Algorithm 1 relies on the classic algorithm for the optimal path in a topologically-sorted DAG. The score for a walk $\pi = (j_0, j_1, \ldots, j_{L-1}, j_L)$ in $G_c$ is

$$\left[ \prod_{\ell=1}^{L} \frac{\partial x_{j_\ell}^{(\ell)}}{\partial x_{j_{\ell-1}}^{(\ell-1)}} \right] \cdot x_{j_0} \tag{40}$$

which is exactly the GNN-GI attribution in [34]. By replacing the gradient matrices $\frac{\partial x_u^{(\ell)}}{\partial x_{u'}^{(\ell-1)}}$ with LRP matrices, one can obtain the exact most relevant walk by LRP score.

Since $G_c$ has $O(L|V|)$ nodes and $O(L|E|)$ edges, Algorithm 1 performs $O(L(|V| + |E|)$ matrix multiplications. Thus the runtime is $O(L(|V| + |E|)\gamma(\bar{p}))$, where $\gamma(\bar{p}))$ is the runtime of any matrix multiplication algorithm. □

**Algorithm 1** DAG-longest path for top relevant walk

**Require:** $G_c$, $\frac{\partial x_u^{(\ell)}}{\partial x_{u'}^{(\ell-1)}}$, $x_u^{(\ell)}$ for $\ell = 1, \ldots, L$

```
parent(u^(ℓ)) ← None for each u^(ℓ)
grad_cum(u^(ℓ)) ← None for each u^(ℓ), ℓ = 0, ..., L − 1
score(u^(ℓ)) ← −∞ for each u^(ℓ), ℓ = 0, ..., L − 1
grad_cum(u^(ℓ)), score(u^(L)) ← ∂z/∂x_u^(L) for each u^(L)
for ℓ = L, ..., 1 do
    for u^(ℓ) ∈ N^(ℓ)(v) do
        for (u'^(ℓ−1), u^(ℓ)) ∈ E^(ℓ)(G_c) do
            new_score ← grad_cum(u^(ℓ)) ∂x_u^(ℓ)/∂x_u'^(ℓ−1) x_u^(ℓ−1)
            if new_score < score(u'^(ℓ−1)) then
                parent(u'^(ℓ−1)) ← u^(ℓ)
                score(u'^(ℓ−1)) ← new_score
                grad_cum(u'^(ℓ−1)) ← grad_cum(u^(ℓ)) ∂x_u^(ℓ)/∂x_u'^(ℓ−1)
u*^(0) ← argmax_{u^(0)} score(u^(0))
Traverse parents from u*^0 for the most relevant path π*
return π*
```

## A.5 Proof of Proposition 5.3

Proof. We expand (24) by writing

$$\frac{\partial (x_{j_\ell})_{k_\ell}}{\partial (x_{j_{\ell-1}})_{k_{\ell-1}}} = P_{j_\ell, k_\ell}^{(\ell)} \omega_{j_{\ell-1}, j_\ell}^{(\ell)} W_{k_{\ell-1}, k_\ell}^{(\ell)}. \tag{41}$$

Upon noticing that

$$\omega_{j_{\ell-1}, j_\ell}^{(\ell)} = A_{j_{\ell-1}, j_\ell}^{(\ell)} \tag{42}$$

we obtain (22), completing the proof. □

# B Analysis of Regularization Terms in GNNExplainer

Here we analyze the effect of the regularization terms $\alpha \|\Omega\|_1$ and $\beta H(\Omega)$ in GNNExplainer's optimization objective (5). We begin by analyzing the size term $\alpha \|\Omega\|$, which is straightforward. For each edge weight $\omega_{ij}$ we have

$$\frac{\partial}{\partial \omega_{ij}} \alpha \|\Omega\|_1 = \frac{\partial}{\partial \omega_{ij}} \alpha \sum_{(u,u') \in E} \omega_{ij} = \alpha \tag{43}$$

for any value of $\omega_{ij}$, since each $\omega_{u,u'} \geq 0$.

To analyze the entropy term, we first briefly discuss the initialization strategy for GNNExplainer. Both the default parameters of the official implementation given by [46] and the implementation

| | GCN | | | | | GAT | | | | | GIN | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **Layers** | 1 | 2 | 3 | 4 | 5 | 1 | 2 | 3 | 4 | 5 | 1 | 2 | 3 | 4 | 5 |
| Cora | 83.50 | 82.70 | 83.30 | 81.69 | 82.90 | 83.70 | 82.70 | 81.49 | 82.09 | 83.30 | 82.70 | 82.70 | 79.88 | 81.49 | 82.49 |
| Citeseer | 68.62 | 68.92 | 67.72 | 67.27 | 69.07 | 69.67 | 70.57 | 69.82 | 70.27 | 68.32 | 69.97 | 67.87 | 67.27 | 67.87 | 67.57 |
| Cornell | 48.64 | 48.64 | 37.83 | 37.83 | 48.64 | 48.64 | 48.64 | 54.05 | 37.83 | 40.54 | 43.24 | 48.64 | 48.64 | 43.24 | 43.24 |
| Texas | 45.94 | 32.43 | 37.83 | 32.43 | 43.24 | 54.05 | 35.13 | 48.64 | 27.03 | 64.86 | 32.43 | 24.32 | 37.84 | 35.14 | 40.54 |
| Wisconsin | 49.01 | 43.13 | 39.21 | 43.13 | 43.13 | 60.78 | 50.98 | 39.22 | 45.10 | 56.86 | 45.10 | 33.33 | 27.45 | 29.41 | 41.18 |
| MUTAG | 69.23 | 63.46 | 73.08 | 73.08 | 73.08 | 73.08 | 65.38 | 65.38 | 65.38 | 61.54 | 84.62 | 76.92 | 88.46 | 76.92 | 73.08 |
| PROTEINS | 55.78 | 64.41 | 69.52 | 70.09 | 67.11 | 64.44 | 72.56 | 71.09 | 58.69 | 50.00 | 59.29 | 75.03 | 73.06 | 73.09 | 74.70 |
| Graph-SST2 | 80.76 | 83.64 | 87.38 | 88.19 | 85.29 | 83.35 | 81.89 | 84.84 | 50.00 | 50.00 | 82.13 | 87.64 | 87.43 | 88.30 | 88.19 |

**Table 1: Accuracy (node classification) or AUROC (graph classification) of trained models on each dataset in Section 6.2**

given in PyTorch Geometric [13] use Kaiming He initialization [15] composed with a sigmoid function to map the result to $[0, 1]$. Hence at $t = 0$,

$$\mathbb{E}\left[\omega_{ij\,(t=0)}\right] = \frac{1}{2}. \tag{44}$$

Then a standard calculation gives

$$\frac{\partial}{\partial \omega_{ij}} H(\Omega) = -\frac{1}{|E|} \log\left(\frac{\omega_{ij}}{1-\omega_{ij}}\right) \tag{45}$$

so in expectation, we have at initialization that

$$\mathbb{E}\left[\frac{\partial H(\Omega)}{\partial \omega_{ij}}\Big|_{\omega_{ij}=\omega_{ij\,(t=0)}}\right] = 0. \tag{46}$$

Thus, in expectation, the term $\beta H(\Omega)$ has no effect at initialization. However, if $\omega_{ij} > 1/2$, we see that (45) becomes negative. Similarly, if $\omega_{ij} < 1/2$, (45) becomes positive. Thus, in expectation, the effect of the entropy term will be determined by the first step of gradient descent, which is determined by

$$-\eta\left(\frac{\partial}{\partial \omega_{ij}} \mathrm{CE}(Y_v, \Phi(v; G_S(\Omega))) + \alpha\right) \tag{47}$$

where $\eta$ is the learning rate. If we have

$$\frac{\partial}{\partial \omega_{ij}} \mathrm{CE}(Y_v, \Phi(v; G_S(\Omega))) + \alpha < 0 \tag{48}$$

then to minimize (5), GNNExplainer will take a step towards 1. In expectation, this means that $\omega_{ij\,t=1} > 1/2$ and so

$$\frac{\partial}{\partial \omega_{ij}} H(\Omega)|_{\omega_{ij}=\omega_{ij\,t=1}} = -\frac{1}{|E|} \log\left(\frac{\omega_{ij\,t=1}}{1-\omega_{ij\,t=1}}\right) < 0. \tag{49}$$

Similarly, if

$$\frac{\partial}{\partial \omega_{ij}} \mathrm{CE}(Y_v, \Phi(v; G_S(\Omega))) + \alpha > 0 \tag{50}$$

then

$$\frac{\partial}{\partial \omega_{ij}} H(\Omega)|_{\omega_{ij}=\omega_{ij\,t=1}} > 0 \tag{51}$$

as well. Thus under the conditions analogous to Proposition 4.2, where the expression

$$\frac{\partial}{\partial \omega_{ij}} \mathrm{CE}(Y_v, \Phi(v; G_S(\Omega))) + \alpha \tag{52}$$

is the same sign for all values of $\omega_{ij}$, the entropy regularization merely accelerates the gradient descent towards 0 or 1.

# C Training Details

We conducted our experiments using PyTorch [29] and PyTorch Geometric [13] on NVIDIA RTX A6000 GPUs.

## C.1 Synthetic Data

For all synthetic experiments we use a GraphSAGE network [14] with sum aggregation, ReLU activations, and no subsampling of node neighborhoods.

*Negative Evidence.* We train a 1-layer network with no bias terms and no root weight using the Adam optimizer [19] with a learning rate of $\eta = 0.01$ and $L_1$ regularization ($\gamma = 0.01$) over 1,000 epochs. Here $L_1$ regularization encourages the network to learn the ground truth. For GNNExplainer, we set the entropy regularization to $\beta = 0$, as all explanations will be connected. To more closely match the scenario in Section 4.1, we use a smaller edge regularization of $\alpha = 0.001$ and a learning rate of 0.5 to ensure GNNExplainer reaches convergence. The model reaches 100% accuracy on the test set.

*Infection.* We train a 4-layer sum-aggregated GraphSAGE with ReLU activation and a hidden layer width of 20. We use Adam for 100 epochs with a learning rate of $\eta = 0.005$ and weight decay ($L^2$ regularization) with $\gamma = 3 \times 10^{-4}$. The model achieves perfect accuracy on the test graph. For GNNExplainer, we use the default parameters provided by PyTorch Geometric [13], training over 100 epochs with a learning rate of $\eta = 0.003$ and hyperparaeters $\alpha = 0.005$ and $\beta = 1$.

## C.2 Real Data

Each GNN has a hidden width of 32 and is trained using Adam ($\eta = 0.003$) using a dropout of 0.5. For the GATs we use 4 attention heads and the GATv2 described in [8]. For datasets with edge features, we augment GIN with the GINE layers from [16]. For node classification (Cora, Citeseer, Cornell, Texas, Wisconsin) we train for 1000 epochs with weight decay of $\gamma = 10^{-5}$. For MUTAG and PROTEINS we train for 2000 epochs with weight decay of $\gamma = 10^{-4}$ and a batch size of 32. For Graph-SST2, we train for 100 epochs using a batch size of 128 and $\gamma = 10^{-6}$. Each graph classification model uses mean pooling to produce the final graph-level output, as well as layer normalization. Table 1 shows the test performance of each model. For node classification we report accuracy. The graph classification tasks are binary, so we report AUROC.

Received 10 February 2025